

A Simple APB Bus Interface for a Timer

Introduction

The Advanced Peripheral Bus (APB) is part of ARM's AMBA protocol family and is widely used for simple, low-power peripherals such as timers, UARTs, GPIO, and watchdogs. It is designed for low-bandwidth communication and provides a straightforward register-based interface between a processor and peripheral logic. In this lab, the APB protocol is used to build a simple timer peripheral that can be programmed, started, and monitored through memory-mapped registers.

Objective

The objective of this lab is to design and verify a simple APB slave interface for a timer peripheral. The timer is controlled through APB-accessible registers that allow software to load a countdown value, start or stop the timer, and read its completion status. Through this design, candidates will learn how APB handshaking works, how peripheral registers are mapped into memory space, and how bus transactions interact with internal timer logic during simulation.

APB Basics

The APB interface uses a small set of control and data signals to communicate with peripherals. PCLK is the APB clock that drives all bus activity, and PRESETn is the active-low reset signal used to initialize the peripheral. PADDR specifies the address of the register being accessed, while PWDATA carries data written from the bus to the peripheral. PRDATA carries data read back from the peripheral to the bus. PWRITE indicates the transfer direction, where 1 represents a write and 0 represents a read. PSEL selects the peripheral for the transaction, and PENABLE indicates the access phase of the APB transfer.

APB transactions occur in two phases. In the Setup Phase, PSEL is asserted, PENABLE remains low, and the address and control signals become valid. In the Access Phase, PENABLE is asserted, and the actual read or write transfer takes place. This two-phase protocol makes APB much simpler than high-performance buses such as AHB or AXI, since it does not support pipelining, burst transfers, or out-of-order transactions.

Block Diagram

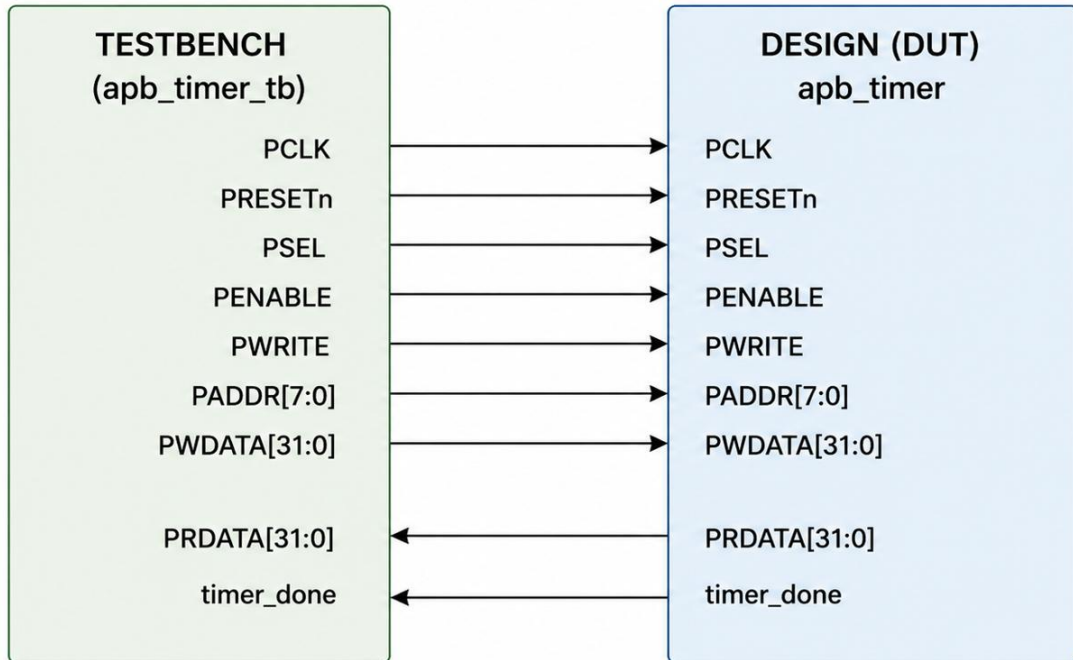


Fig: Block Diagram of APB Timer

Procedure To Create Lab:

Step 1: Set Up Project Directory

- 1.1: Create directory for project: `mkdir apb_timer`
- 1.2: Open project directory: `cd apb_timer`

Step 2: Create Design File

- 2.1: Open gvim editor for design with command: `gvim apb_timer.v`

RTL Design Code:

```
// APB-based countdown timer
// Register map:
// 0x00 -> Load register
// 0x04 -> Control register (bit[0] : start/stop)
// 0x08 -> Status register (bit[0] : timer_done)

`timescale 1ns/1ps

module apb_timer #(
    parameter WIDTH = 8    // Counter width
) (
```

```
// APB interface
input wire      PCLK,          // APB clock
input wire      PRESETn,       // Active-low reset
input wire      PSEL,          // Peripheral select
input wire      PENABLE,       // Access phase indicator
input wire      PWRITE,        // 1 = write, 0 = read
input wire [7:0] PADDR,        // Register address
input wire [31:0] PWDATA,       // Write data
output reg [31:0] PRDATA,       // Read data

// Timer status output
output reg      timer_done

);

// Internal timer registers
reg [WIDTH-1:0] load_val;       // Programmed timer value
reg [WIDTH-1:0] count;          // Active counter
reg              running;       // Timer enable

// APB write path
// Valid write: PSEL=1, PENABLE=1, PWRITE=1
always @(posedge PCLK or negedge PRESETn) begin
    if (!PRESETn) begin
        load_val <= 0;
        running <= 0;
    end
    else if (PSEL && PENABLE && PWRITE) begin
        case (PADDR)
            8'h00: load_val <= PWDATA[WIDTH-1:0];
            8'h04: running <= PWDATA[0];
        endcase
    end
end

// APB read path
// Valid read: PSEL=1, PWRITE=0
always @(*) begin
    PRDATA = 32'h0;
    if (PSEL && !PWRITE) begin
        case (PADDR)
            8'h00: PRDATA = {{(32-WIDTH){1'b0}}, load_val};
            8'h04: PRDATA = {31'b0, running};
        endcase
    end
end
```

```
8'h08: PRDATA = {31'b0, timer_done};
default: PRDATA = 32'h0;
endcase
end
end

// Timer logic
// Counter decrements while running
// On reaching zero, timer_done is asserted
always @(posedge PCLK or negedge PRESETn) begin
    if (!PRESETn) begin
        count        <= 0;
        timer_done <= 0;
    end
    else if (running) begin
        if (count == 0) begin
            count        <= load_val;
            timer_done <= 1;
            running      <= 0;
        end
        else begin
            count        <= count - 1;
            timer_done <= 0;
        end
    end
end
end
endmodule
```

Step 3: Create Testbench File

3.1: Open gvim editor for testbench with command: gvim apb_timer_tb.v

Testbench:

```
`timescale 1ns/1ps
module apb_timer_tb;

    // APB interface signals
    reg        PCLK;
    reg        PRESETn = 0;
    reg        PSEL    = 0;
    reg        PENABLE = 0;
    reg        PWRITE  = 0;
```

```
reg [7:0] PADDR = 0;
reg [31:0] PWDATA = 0;
wire [31:0] PRDATA;
wire timer_done;

// DUT instance
apb_timer uut (
    .PCLK(PCLK),
    .PRESETn(PRESETn),
    .PSEL(PSEL),
    .PENABLE(PENABLE),
    .PWRITE(PWRITE),
    .PADDR(PADDR),
    .PWDATA(PWDATA),
    .PRDATA(PRDATA),
    .timer_done(timer_done)
);

// Clock generation
initial PCLK = 0;
always #5 PCLK = ~PCLK;

// Monitor timer completion
always @(posedge timer_done)
    $display("Time=%0t : TIMER DONE", $time);

// APB write task
task apb_write(input [7:0] addr, input [31:0] data);
begin
    @(posedge PCLK);
    PSEL = 1; PWRITE = 1; PENABLE = 0; PADDR = addr;
    PWDATA = data;
    @(posedge PCLK);
    PENABLE = 1;
    @(posedge PCLK);
    PSEL = 0; PENABLE = 0; PWRITE = 0;
end
endtask

// APB read task
task apb_read(input [7:0] addr);
begin
    @(posedge PCLK);
```

```
PSEL = 1; PWRITE = 0; PENABLE = 0; PADDR = addr;
@(posedge PCLK);
PENABLE = 1;
@(posedge PCLK);
$display("Read [0x%0h] = %0d", addr, PRDATA);
PSEL = 0; PENABLE = 0;
end
endtask

// Test sequence
initial begin
    $dumpfile("apb_timer.vcd");
    $dumpvars(0, apb_timer_tb);

    // Apply reset
    #10 PRESETn = 1;

    // Program load value = 5
    apb_write(8'h00, 5);

    // Start timer
    apb_write(8'h04, 1);

    // Wait and read status
    #80 apb_read(8'h08);

    // Program load value = 3
    apb_write(8'h00, 3);

    // Start timer again
    apb_write(8'h04, 1);

    // Read status delay
    #50 apb_read(8'h08);

    #50 $finish;
end

endmodule
```

3.2: Save the testbench file using command: :wq

3.3: Check the saved file in current directory: use command ls

Step 4: Simulate the Design with Verilator

a) Create a script file `run.sh`

```
#!/bin/bash
```

```
# Use Verilator to compile and generate simulation files
```

```
verilator --binary -j 0 -Wall \  
  apb_timer.v apb_timer_tb.v \  
  --top apb_timer_tb --timing --CFLAGS "-std=c++20" --trace
```

```
# Step 2: Enter the object directory
```

```
cd obj_dir || { echo "obj_dir not found"; exit 1; }
```

```
# Step 3: Compile simulation binary
```

```
make -f Vapb_timer_tb.mk Vapb_timer_tb || { echo "Compilation  
failed"; exit 1; }
```

```
# Step 4: Run the simulation
```

```
./Vapb_timer_tb || { echo "Simulation failed"; exit 1; }
```

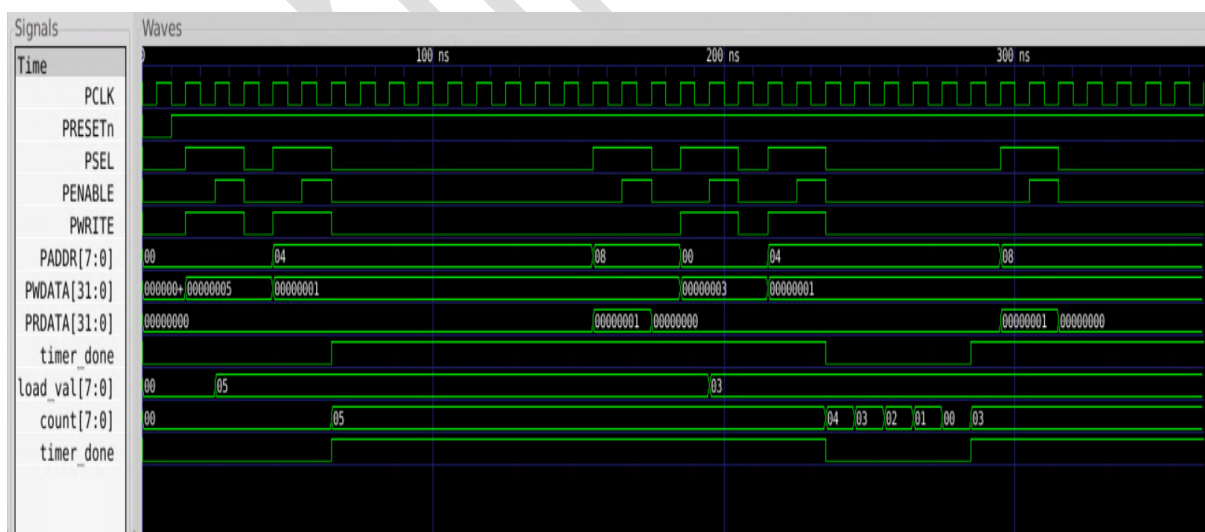
```
# Step 5: Launch GTKWave to view the waveform
```

```
gtkwave apb_timer.vcd
```

b) Make the script executable: `chmod +x run.sh`

c) Run the simulation and observe waveform : `./run.sh`

GTKWave Waveform



Waveform Observations

In GTKWave, observe the APB handshake signals PSEL, PENABLE, and PWRITE to understand how APB transactions are sequenced across setup and access phases. Monitor the address and data lines PADDR, PWDATA, and PRDATA to verify correct register access during read and write operations. Internal peripheral signals such as load_val, count, running, and timer_done should also be observed to track the timer's functional behavior. When a write occurs to address 0x00, the load_val register updates with the programmed countdown value. When a write occurs to address 0x04 with value 1, the timer is enabled by asserting running. Once enabled, the counter begins its countdown operation. When count reaches 0, the timer_done signal is asserted, indicating timer completion. A read from address 0x08 returns the current status of the timer_done flag.

Conclusion

This lab demonstrates that the APB protocol operates in two phases: Setup and Access. It also shows that simple peripherals can be controlled using minimal APB interface logic and basic address decoding. The design illustrates how control, status, and data registers are mapped into a memory-mapped I/O space, allowing software to interact with hardware through standard register accesses. By integrating the timer with an APB interface, candidates learn how an APB bus controls peripheral hardware, how registers are exposed through memory-mapped addressing, and how software-visible registers interact with internal timer logic. The simulation further helps in understanding how to observe, debug, and verify a complete bus-connected peripheral in a waveform environment.